



CLOUDSTACK PROJECT
LXD | DOCKER | CLOUDPANEL
IMPLEMENTATION DOCUMENTATION 2026

Projekt - Dokumentation

Sandbox für Docker- und Web-Anwendungen

Projektzeitraum: 13.04.2026 - 17.04.2026

Ingo Weber
Ausbildungsberuf: IT-Systemelektroniker

Inhaltsverzeichnis

1	Einleitung / Informationen	1
1.1	Die pädagogische und technische Problemstellung	1
1.2	Architektur und Sicherheitskonzept	1
1.3	Ressourceneffizienz unter Legacy-Bedingungen	1
2	Ist - Zustand	2
3	Soll - Zustand	2
4	Projektplanung	3
4.1	Projektstrukturplan	3
4.2	Zeit- und Ressourcenplanung	4
5	Durchführung	5
5.1	Detailplanung / Konzept	5
5.2	Umsetzung / Implementierung	5
5.2.1	Recovery-System und Partitionierung	5
5.2.2	Haupt - System	6
5.3	Qualitätssicherung (Testfälle / Abnahme)	8
6	Projektabschluss	9
6.1	Soll - Ist - Vergleich	9
6.2	Fazit / Zusammenfassung	9
A	Anlagen	I
B	Anlagen - Diagramme und Screenshots	II
B.1	Projektphasen und Zeitplanung (Wasserfallmodell)	II
B.2	Tabellen	III
B.3	Bildschirmaufnahmen / Fotos	III
C	Anlagen - Quelltexte	V
C.1	Beispiel-Custom Grub-Konfiguration (40_custom)	V
C.2	Konfiguration von ttyd des Hauptsystems	V
C.3	Cronjob des Root-Nutzers	V
C.4	Auto-Healer-Script (nginx-autohealer.sh)	VI
C.5	Service-Script für den Factory-Reset (sda2-factory-reset.service)	IX
C.6	Shell-Script für den Factory-Reset	IX
C.7	Standard-Konfiguration der LXC-Container (default-profil.yaml)	X
C.8	Konfigurationsdatei der Netzwerkbrücke (networks-lxdbr0).yaml)	X
C.9	Konfigurationsdatei des Docker-Containers (docker-sandbox.yaml)	X
C.10	Konfiguration von ttyd des LXC-Containers	XI
C.11	Ausschnitt der PHP-Datei des Dashboards (index.php)	XII
C.12	Ausschnitt der Design-Anpassungen (CSS)	XIII
C.13	vHost des Reverse-Proxies Hauptsystem-TTYd	XV
C.14	vHost des Reverse-Proxies Container-TTYd	XVI
D	Anlagen - Hostnamen, IP-Adressen, Initial-Passwörter	XVII
	Quellenverzeichnis	XVIII

1 Einleitung / Informationen

Das vorliegende Projekt widmet sich der Konzeption, Implementierung und systemseitigen Härtung einer hochresilienten Server-Infrastruktur. Primäres Ziel war die Schaffung eines praxisnahen Lernortes (Sandbox), der speziell für den Einsatz im schulischen Kontext optimiert ist. Die Herausforderung bestand darin, eine Umgebung zu entwickeln, die einerseits technologische Tiefe bietet und andererseits immun gegen Fehlkonfigurationen oder destruktive administrative Eingriffe bleibt.

1.1 Die pädagogische und technische Problemstellung

In der modernen IT-Ausbildung ist der Übergang von der Theorie zur Praxis oft durch Sicherheitsbedenken gehemmt. Für einen nachhaltigen Lernerfolg ist es jedoch unerlässlich, dass Lernende administrative Tätigkeiten – wie die Konfiguration komplexer Webserver-Stacks oder die Orchestrierung von Container-Technologien – unter Realbedingungen erproben können.

Dies führt zu einem inhärenten Konflikt:

- **Anforderung:** Vergabe weitreichender Root-Rechte an „administrative Laien“ zur Förderung der Explorationsfreiheit.
- **Risiko:** Systeminstabilität, Datenverlust oder die Kompromittierung des Host-Systems durch unsachgemäße Bedienung.

1.2 Architektur und Sicherheitskonzept

Um diesen Widerspruch aufzulösen, demonstriert das Projekt einen Lösungsansatz, der auf drei zentralen Säulen basiert:

- **Asymmetrische Systemarchitektur:** Eine strikte Trennung zwischen der Verwaltungsebene (Host) und der Nutzer-Ebene (Gäste). Dies stellt sicher, dass kritische Systemressourcen für die Lernenden zu keinem Zeitpunkt direkt erreichbar sind.
- **Strikte Isolationsschichten:** Durch den Einsatz moderner Virtualisierungs- oder Containerisierungsverfahren werden die Experimentier-Umgebungen so voneinander und vom Kernsystem gekapselt, dass „Break-outs“ effektiv verhindert werden.
- **Autonomes Auto-Healing:** Ein wesentliches Merkmal der Infrastruktur ist die Fähigkeit zur Selbstheilung. Durch automatisierte Rollback-Mechanismen oder die Instanziierung von „Wegwerf-Umgebungen“ kann das System nach einer Fehlkonfiguration autonom und ohne manuellen Eingriff eines Administrators in einen definierten Ausgangszustand zurückkehren.

1.3 Ressourceneffizienz unter Legacy-Bedingungen

Ein besonderer Fokus lag auf der extremen Optimierung der Systemressourcen. Das Projekt beweist, dass hochmoderne Infrastruktur-Konzepte nicht zwingend High-End-Hardware erfordern. Als physisches Fundament diente ein betagter Intel Core i5-3450 mit lediglich 8 GB RAM.

Durch eine rigorose Auswahl schlanker Software-Stacks und eine effiziente Speicherverwaltung wurde sichergestellt, dass trotz der Hardware-Limitierungen eine flüssige, mehrbenutzerfähige Umgebung bereitgestellt werden kann, die den Anforderungen eines modernen IT-Unterrichts vollumfänglich entspricht.

2 Ist - Zustand

In klassischen schulischen IT-Umgebungen besteht ein inhärenter Zielkonflikt: Werden den Schülern weitreichende Rechte, wie Shell-Zugriff oder Berechtigungen zur Modifikation von Webserverkonfigurationen, eingeräumt, kommt es durch Fehlbedienungen (Copy-Paste-Fehler", versehentliches Löschen von Systempfaden) unweigerlich zu kritischen Systemausfällen. Ein einziger fehlerhafter Syntax-Eintrag in einer Webserver-Konfiguration durch einen Nutzer führt nach einem Neustart des Daemons zum Ausfall aller anderen auf dem Server gehosteten Schüler-Projekte. Bisherige Lösungsansätze basierten entweder auf starker Restriktion (was den Lerneffekt minimiert) oder auf hohem administrativen Overhead für die Verantwortlichen, die das System nach Fehlern manuell debuggen oder aus Backups wiederherstellen mussten. Zudem erforderte eine vollständige Virtualisierung jedes Schüler-Arbeitsplatzes Hardware-Ressourcen, die in dem vorliegenden Szenario (8 GB RAM) physisch nicht abgebildet werden können.

3 Soll - Zustand

Ziel des Projekts ist die Implementierung eines fehlertoleranten, sich selbst korrigierenden Systems (Trampolin-Prinzip). Der Soll-Zustand definiert sich durch folgende Kernanforderungen:

- Hardware-Effizienz: Das primäre Web-Hosting-Stack (CloudPanel, Nginx, PHP, MySQL) muss nativ auf dem Bare-Metal-Host (Ubuntu 24.04 LTS) betrieben werden, um Virtualisierungs-Overhead zu vermeiden und die Leistung auf der limitierten Hardware zu maximieren.
- Gezielte Isolation: Experimentelle oder potenziell ressourcenfressende Applikationen (z.B. Docker-Engines) dürfen nicht auf dem Host laufen, sondern müssen in leichtgewichtige Systemcontainer (LXC via LXD) ausgelagert werden.
- Autonomes Auto-Healing: Das System muss in der Lage sein, fatale Konfigurationsfehler oder destruktive Dateioperationen der Nutzer autonom zu erkennen. Ein Watchdog-Skript soll defekte Konfigurationen isolieren und fehlende Verzeichnisstrukturen (Superset) selbstständig regenerieren, um Host-Crashes präventiv abzuwenden.
- Zentrale Telemetrie und Recovery: Es muss ein zentrales Web-Dashboard (Infrastructure Hub) geschaffen werden, das Administratoren in Echtzeit die Systemlast anzeigt, Fehlerzustände visualisiert und eine One-Click-Wiederherstellung von Containern (Soft Reset) oder der gesamten Host-Partition (Hard Reset via partclone) ermöglicht.

4 Projektplanung

4.1 Projektstrukturplan

Der Projektstrukturplan unterteilt die Realisierung des Infrastructure Hubs in fünf aufeinanderfolgende Kernphasen, welche die asymmetrische Systemarchitektur von der ersten Hardware-Analyse bis zur finalen System-Sterilisierung abbilden. Die erste Phase, **Analyse und Konzeption**, befasste sich

primär mit der Evaluation der physischen Hardware-Limits des Intel Core i5-3450 und der daraus resultierenden Entscheidung für einen hybriden Architekturansatz. Hierbei wurden das Partitionsschema für den Host und das Recovery-System (sda3/sda2) definiert sowie die Netzwerkplanung vorgenommen, um mittels eines MACVLAN-Bypasses die Erreichbarkeit der isolierten Dienste für das Monitoring sicherzustellen. In der zweiten Phase, dem **Infrastruktur-Setup**, erfolgte die Grundinstallation von

Ubuntu 24.04 LTS auf der Hauptpartition. Um die I/O-Performance und den RAM-Verbrauch zu optimieren, wurde der primäre Service-Stack bestehend aus CloudPanel, Nginx und MariaDB nativ auf dem Host-System implementiert. Parallel dazu wurde der Virtualisierungslayer via LXD konfiguriert und die isolierte **docker-sandbox** zur Kapselung potenziell instabiler Drittanwendungen eingerichtet. Den Abschluss dieser Phase bildete die Installation des minimalen Zweit-OS auf der Recovery-Partition sda2. Die dritte Phase, die **Entwicklung des Immunsystems**, stellt den technologischen Kern des

Projekts dar. Hierbei wurde das autonome Bash-Skript **nginx-healer.sh** programmiert, welches über eine Closed-Loop-Logik verfügt, um syntaktische Fehler in vHost-Konfigurationen zu isolieren und gelöschte Systempfade durch eine Superset-Regeneration wiederherzustellen. Die Integration in das Sicherheitskonzept erfolgte durch präzise definierte Ausnahmen in der **/etc/sudoers** sowie die Automatisierung über Systemd-Overrides und Cronjobs. In der vierten Phase wurde das **Dashboard**

und die Telemetrie-Einheit realisiert. Das Frontend wurde mittels PHP und TailwindCSS entwickelt, während das Backend über eine asynchrone AJAX-Schnittstelle Echtzeit-Metriken zu CPU-Last, RAM-Belegung und Prozessortemperaturen bereitstellt. Zudem wurde die grafische Benutzeroberfläche für die Recovery-Trigger implementiert, welche die Durchführung von Soft- und Hard-Resets direkt über das Web-UI ermöglicht. Die finale fünfte Phase umfasste die **Validierung und den Projekt-**

abschluss. Mittels Greybox-Testing (Chaos-Monkey-Prinzip) wurden kritische Ausfallszenarien simuliert, um die Resilienz des Auto-Healers und der Recovery-Pfade unter Beweis zu stellen. Nach der Bereinigung aller Systemprotokolle und der Behebung letzter Boot-Fehlermeldungen wurde das sterile **clean-state** Image via **Partclone** auf sda2 gesichert. Den formalen Abschluss bildete die Erstellung der technischen Dokumentation sowie der Kundendokumentation für Webentwickler.

4.2 Zeit- und Ressourcenplanung

Wie in Anlage B.1 auf Seite II als Diagramm zu sehen ist.

Phase 1: Analyse und Definition (6 Stunden)

- Ist-Analyse der bestehenden Infrastruktur (1h)
- Aufnahme der funktionalen Anforderungen der Entwicklungsabteilung (2h)
- Erstellung des Lastenhefts (1h)
- Wirtschaftlichkeitsbetrachtung und Vergleich der Virtualisierungsmethoden (2h)

Phase 2: Planung und Design (6 Stunden)

- Auswahl und Spezifikation der Hardwarekomponenten (2h)
- Entwurf der Systemarchitektur
(Schichtenmodell: Hardware -> OS -> CloudPanel -> LXC -> Docker) (2h)
- Erstellung eines Sicherheits- und Backup-Konzepts (2h)

Phase 3: Realisierung und Implementierung (16 Stunden)

- Hardware-Montage, BIOS-Konfiguration und Installation des Basis-Betriebssystems (4h)
- Installation und Konfiguration von CloudPanel (2h)
- Einrichtung von LXD und Erstellung der System-Container-Profile (4h)
- Implementierung der Docker-Umgebung innerhalb der LXC-Struktur (3h)
- Installation und Konfiguration von Portainer zur Container-Verwaltung (3h)

Phase 4: Test und Abnahme (4 Stunden)

- Durchführung von Funktionstests (Deployment einer Beispiel-Applikation) (1,5h)
- Validierung der Ressourcen-Limits und der Systemisolation (1,5h)
- Projektabnahme (1h)

Phase 5: Dokumentation (8 Stunden)

- Erstellung der fachspezifischen Projektdokumentation (6h)
- Erstellung einer Kundendokumentation (Bedienungsanleitung für Entwickler) (2h)

5 Durchführung

5.1 Detailplanung / Konzept

Hardware siehe Tabelle 1 auf Seite III.

Betriebskonzept und Energiemanagement Ein wesentlicher Aspekt des stabilen Dauerbetriebs am Einsatzort ist die Koordination mit der lokalen Stromversorgung. Da im Lernort zum Betriebsende die Stromzufuhr zentral deaktiviert wird, wurde ein zweistufiges Verfahren implementiert:

- **Kontrolliertes Herunterfahren (Software):** Ein konfigurierter Cronjob führt arbeitstäglich vor der Stromabschaltung den Befehl `sudo poweroff` aus. Dies stellt sicher, dass alle laufenden LXD-Container und Docker-Instanzen ihre Schreibvorgänge abschließen und das Dateisystem sauber ausgehängt wird, um Datenverlust oder Inkonsistenzen zu vermeiden.
- **Automatischer Kaltstart (Hardware):** Durch die UEFI-Einstellung „*Restore on AC/Power Loss*“ auf „*Power On*“ wird sichergestellt, dass der ThinClient am nächsten Morgen bei Wiederherstellung der Stromzufuhr autonom startet, ohne dass ein manuelles Eingreifen des Personals erforderlich ist. (siehe Abbildung 2 auf Seite IV)

5.2 Umsetzung / Implementierung

Die Initialisierung des Projekts begann mit der Vorbereitung des Installationsmediums sowie der Hardware-Konfiguration des Zielsystems. Dieser Prozess gliedert sich in die folgenden Schritte:

1. **Bereitstellung des Installationsmediums:** Zunächst wurde das offizielle ISO-Abbild von Ubuntu Server in der aktuellsten Version bezogen [1]. Zur Erstellung des bootfähigen Mediums wurde das Open-Source-Tool *Ventoy* verwendet. Im Gegensatz zu herkömmlichen Imaging-Verfahren erlaubt Ventoy das direkte Starten von ISO-Dateien durch einfaches Kopieren auf den Datenträger, was eine flexible Handhabung verschiedener Installationsmedien ermöglicht.
2. **Hardware-Konfiguration (UEFI/BIOS):** Als Zielhardware dient ein ThinClient, dessen Firmware-Einstellungen (BIOS/UEFI) für den Serverbetrieb optimiert werden mussten. Eine zentrale Anforderung für den Headless-Betrieb ist die Hochverfügbarkeit nach einem Stromausfall. Daher wurde im Power-Management des Geräts die Option „*Restore on AC/Power Loss*“ auf „*Power On*“ gestellt. Dies stellt sicher, dass das System autonom startet, sobald die Stromversorgung wiederhergestellt ist, ohne dass ein manueller Eingriff erforderlich ist.

5.2.1 Recovery-System und Partitionierung

Um eine hohe Systemstabilität sowie eine einfache Wiederherstellung im Fehlerfall zu gewährleisten, wurde ein dediziertes Recovery-System implementiert. Die Installation basiert auf der Minimal-Variante von Ubuntu Server, um den Footprint und die potenzielle Angriffsfläche des Wartungssystems gering zu halten.

Installation und Speicherlayout Während der Installation wurde die SSD manuell partitioniert, um das Recovery-System logisch vom späteren Hauptsystem zu trennen und die Integrität der Wartungsumgebung zu sichern. Das Speicherlayout wurde wie folgt strukturiert:

- **sda1:** Bootloader (GRUB) zur zentralen Steuerung des Startvorgangs.
- **sda2:** Recovery-System (20 GB), eingehängt als Root-Verzeichnis (/). Diese Partition beherbergt die notwendigen Werkzeuge und Skripte für den Werksreset.
- **sda3:** Hauptsystem (Restkapazität der SSD), welches in dieser Phase unformatiert bleibt und als Ziel für die spätere Image-Wiederherstellung dient.

Für die initiale Netzwerkanbindung wird DHCP genutzt, wobei der Hostname auf **recovery-sandbox** festgelegt wurde. Zur Fernwartung und Konfiguration wurde bereits während des Setups ein **SSH-Server** aktiviert. Nach der Grundinstallation wurde das System unmittelbar auf den aktuellen Stand gebracht und um die benötigten Werkzeuge für die Partitionssicherung ergänzt:

```
1 sudo apt update && sudo apt dist-upgrade -y
2 sudo apt install -y partclone gzip pv nano
```

Automatisierung des Werksreset Zur Implementierung einer automatisierten Wiederherstellungsroutine wurde eine Systemd-Service-Unit eingerichtet. Dieser Dienst fungiert als Schnittstelle zum Bootvorgang und triggert ein Shell-Skript, welches die automatisierte Rücksetzung des Hauptsystems auf **sda3** übernimmt.

Zunächst wurde die Service-Unit (siehe Anhang C.5) im Verzeichnis */etc/systemd/system/* mit einem Texteditor erstellt:

```
1 sudo nano /etc/systemd/system/factory-reset.service
```

Und anschliessend für den Systemstart aktiviert:

```
1 sudo systemctl enable factory-reset.service
```

Das ausführende Logik-Skript wurde unter */usr/local/bin/factory-restore.sh* hinterlegt. Dieses Skript (siehe Anhang C.6) validiert beim Bootvorgang das Vorhandensein des Recovery-Triggers in der Kernel-Commandline und führt bei Bedarf die Formatierung sowie die Rückspielung des System-Images durch. Um die Ausführbarkeit sicherzustellen, wurde die Datei mit den entsprechenden Rechten versehen:

```
1 sudo chmod +x /usr/local/bin/factory-restore.sh
```

5.2.2 Haupt - System

Installation des Hauptsystems Nachdem die grundlegende Recovery-Infrastruktur auf der Partition **sda2** etabliert wurde, erfolgte im nächsten Schritt die Bereitstellung des eigentlichen Betriebssystems auf der verbleibenden Partition **sda3**. Ziel war es, eine voll funktionsfähige Arbeitsumgebung zu schaffen, die als Basis für spätere Applikationen und Container-Szenarien dient.

Systemkonfiguration und Basis-Setup Die Installation des Hauptsystems basiert ebenfalls auf Ubuntu Server, wurde jedoch im Gegensatz zum Recovery-System als Standard-Installation (Normal-Installation) durchgeführt. Dies stellt sicher, dass alle gängigen Bibliotheken und Werkzeuge für den produktiven Betrieb bereits vorinstalliert sind.

Die Konfiguration während des Setups wurde nach folgenden Parametern vorgenommen:

Partitionierung: Als Zielpartition wurde die zuvor freigehaltene Partition **sda3** gewählt und als Root-Dateisystem (*/*) formatiert.

Netzwerk: Um eine dynamische Integration in die bestehende Laborumgebung zu gewährleisten, wurde die Netzwerkschnittstelle auf den Bezug via DHCP konfiguriert.

Identifikation: Zur eindeutigen Unterscheidung vom Wartungssystem wurde der Hostname auf **sandbox** festgelegt.

Zugriff: Für die Administration über das Netzwerk wurde der OpenSSH-Server bereits während des Installationsdialogs aktiviert.

Integration von Virtualisierung und Containern Ein wesentlicher Bestandteil des Hauptsystems ist die Bereitstellung von Virtualisierungskapazitäten. Hierfür wurde bereits während der Installation die Auswahl des LXD-Snaps getroffen. LXD fungiert als moderner System-Container- und Virtualisierungsmanager, der es ermöglicht, isolierte Umgebungen effizient zu verwalten. Durch die Integration als Snap-Paket wird sichergestellt, dass der Dienst automatisch aktualisiert wird und in einer stabilen, vom restlichen System isolierten Umgebung operiert.

Nach dem ersten Bootvorgang wurde das System finalisiert und der Dienst `ttyd` installiert:

```
1 sudo apt update && sudo apt dist-upgrade -y
2 sudo apt install ttyd
```

Weiter wird dann die Datei `/etc/default/ttyd` wie in Anhang C.2 angepasst und anschließend der Dienst aktiviert und gestartet:

```
1 sudo systemctl enable ttyd
2 sudo systemctl start ttyd
```

Installation von Node.js und CloudPanel Nach der Grundkonfiguration des Hauptsystems erfolgt die Einrichtung der Laufzeitumgebungen und des Hosting-Control-Panels. Ein besonderer Fokus liegt hierbei auf der Kompatibilität der Softwarekomponenten und der initialen Absicherung des Systems.

Laufzeitumgebung Node.js Für die Ausführung moderner Web-Applikationen wird Node.js benötigt. Da CloudPanel aktuell die Versionen 20 und 22 unterstützt, Version 24 jedoch noch nicht freigegeben ist, werden gezielt diese beiden LTS-Versionen (Long Term Support) installiert. Um ein flexibles Management zwischen diesen Versionen zu ermöglichen, wird der *Fast Node Manager* (fnm) eingesetzt. Dieser erlaubt einen schnellen Wechsel der aktiven Version und eine saubere Trennung der Binaries.

Die Installation erfolgt über die offiziellen Quellen mittels der folgenden Befehlskette, die auf der Webseite: <https://nodejs.org/en/download> zu finden ist.

```
1 # Installation des Fast Node Managers (fnm)
2 curl -o- https://fnm.vercel.app/install | bash
3
4 # Installation der benötigten Node.js Versionen
5 fnm install 22
6 fnm install 20
```

CloudPanel Control Panel und administrativer Abschluss Als zentrale Verwaltungsoberfläche für den Serverbetrieb wird **CloudPanel** eingesetzt. Die Wahl fiel auf die Community Edition (CE) in Kombination mit einer leistungsstarken MySQL 8.0 Datenbank-Engine. Der Installationsprozess wird über ein offizielles Skript automatisiert, der auf der Webseite: <https://www.cloudpanel.io/docs/v2/getting-started/other/> zu finden ist, wobei zur Sicherstellung der Datenintegrität vor der Ausführung eine Verifizierung des Skript-Hashwerts erfolgt.

Der Installationsvorgang wird mit folgendem Befehl initiiert:

```
1 # Herunterladen und Verifizieren des CloudPanel-Installers
2 curl -sS https://installer.cloudpanel.io/ce/v2/install.sh -o install.sh
3 echo "19cfa702e7936a79e47812ff57d9859175ea902c62a68b2c15ccd1ebaf36caeb install.sh"
   | sha256sum -c && sudo DB_ENGINE=MYSQL_8.0 bash install.sh
```

Nachdem das Skript die Installation der Pakete und der Datenbank abgeschlossen hat, ist das Panel über den Port 8443 erreichbar. Der abschließende Schritt der Grundinstallation ist die **Einrichtung des Administrator-Kontos**. Da zu diesem Zeitpunkt noch kein Benutzer im System hinterlegt ist,

wird über das Web-Interface der erste privilegierte Benutzer angelegt. Hierbei werden die administrativen Zugangsdaten sowie die Standard-E-Mail-Adresse für Systembenachrichtigungen definiert.

Erst mit der Erstellung dieses Admin-Users ist die Konfiguration der „Sandbox“-Umgebung abgeschlossen und das System befindet sich in einem definierten Zustand, der als Basis für das finale Deployment-Image dient.

Einrichtung von LXD und des LXC-Containers mit der Docker - Engine und Portainer

Healer-Skript / Cron-Jobs

Einrichtung des Dashboards

Einrichtung der Boot-Option für den automatischen Factory-Reset

Konfiguration der OpenSense

Erstellung des Factory-Images

5.3 Qualitätssicherung (Testfälle / Abnahme)

6 Projektabschluss

6.1 Soll - Ist - Vergleich

6.2 Fazit / Zusammenfassung

A Anlagen

Inhaltsübersicht des Anhangs

Diagramme und Screenshots	S. II
--	-------

Enthält das Gantt-Chart zur zeitlichen Stundenverteilung und die Bildschirmaufnahmen.

Quelltexte und Konfigurationen	S. V
---	------

Enthält die Quelltexte sämtlicher Scripte und angelegten Dateien.

Hostnamen, IP-Adressen, Initial-Passwörter	S. XVII
---	---------

Auflistung aller bei der Installation eingerichteten Passwörter, IP-Adressen und FQDNs.

Quellenverzeichnis	S. XVIII
---------------------------------	----------

Vollständige Auflistung aller verwendeten Online-Ressourcen und Dokumentationen.

Online-Repository und Versionsverwaltung

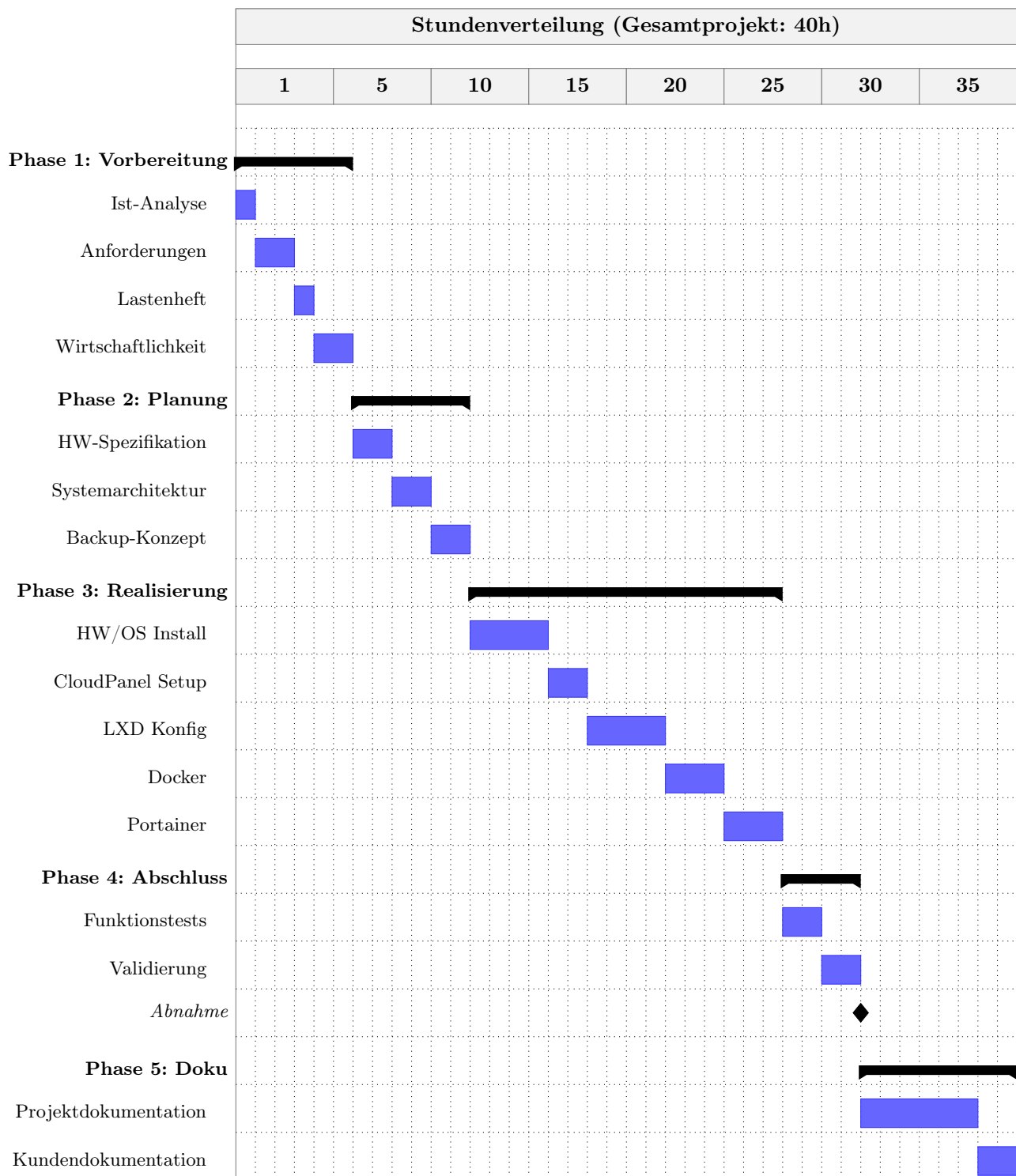
Der vollständige Quellcode, inklusive der Revisionshistorie und der verwendeten Dockerfiles, ist unter folgendem Link öffentlich zugänglich:

`https://github.com/KitoMossman/Sandbox`

Dies ermöglicht eine tiefergehende Prüfung der Implementierungsschritte, die über den Umfang dieser gedruckten Dokumentation hinausgehen. Zu näheren Erklärung liegt auch eine **README.md** vor.

B Anlagen - Diagramme und Screenshots

B.1 Projektphasen und Zeitplanung (Wasserfallmodell)



B.2 Tabellen

Tabelle 1: Hardwarespezifikationen des Zielsystems

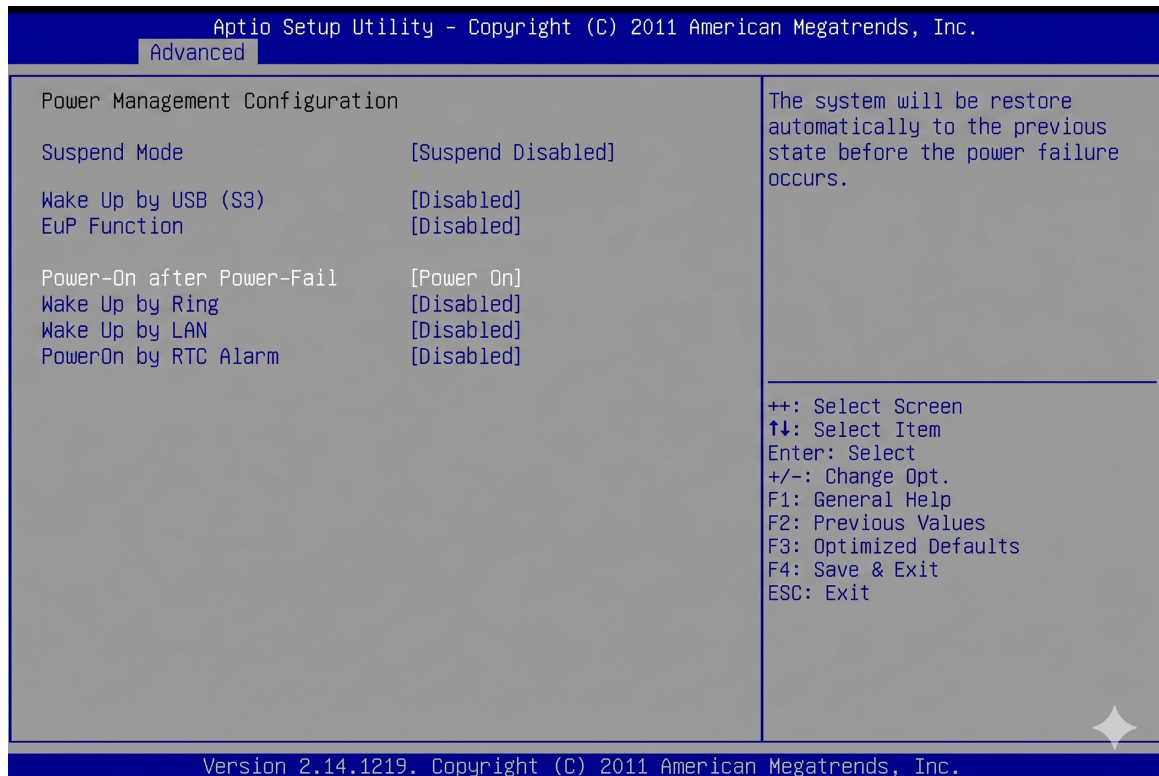
Komponente	Spezifikation
Modell	Terra ThinClient
Prozessor (CPU)	Intel Core i5-3450 (4 Kerne, bis 3,50 GHz)
Arbeitsspeicher	8 GB DDR3 RAM
Festplatte	120 GB SATA SSD
Netzwerk	Gigabit Ethernet
Besonderheit	UEFI mit Auto-Power-On Support

B.3 Bildschirmaufnahmen / Fotos

Abbildung 1: Fotos des Terra Thinclient PCs



Abbildung 2: UEFI - Einstellung



C Anlagen - Quelltexte

C.1 Beispiel-Custom Grub-Konfiguration (40_custom)

```
1 # In /etc/grub.d/40_custom einfügen - UUID von /boot/grub.cfg übernehmen.
2 # Verweist auf das Ubuntu Server Minimal auf /dev/sda2 mit dem hinzugefügtem
   Parameter "factory-reset quiet" was den Service sda2-factory-reset auslöst und
   das Shell-Script "sda2-factory-restore.sh" ausführt, was ohne Nachfrage via
   partclone das Image von /dev/sda3 zurückschreibt.
3
4 menuentry 'FACTORY RESET!' --class ubuntu --class gnu-linux --class gnu --class os
   $menuentry_id_option 'osprober-gnulinix-simple-f2a1bb6d-314e-46bd-92b0-
   f5881f9660d6' {
5     insmod part_gpt
6     insmod ext2
7     set root='hd0,gpt2'
8     if [ x$feature_platform_search_hint = xy ]; then
9         search --no-floppy --fs-uuid --set=root --hint-bios=hd0,gpt2 --hint-efi=hd0
   ,gpt2 --hint-baremetal=ahci0,gpt2 f2a1bb6d-314e-46bd-92b0-f5881f9660d6
10    else
11        search --no-floppy --fs-uuid --set=root f2a1bb6d-314e-46bd-92b0-
   f5881f9660d6
12    fi
13    linux /boot/vmlinuz-6.8.0-100-generic root=UUID=f2a1bb6d-314e-46bd-92b0-
   f5881f9660d6 ro factory-reset quiet
14    initrd /boot/initrd.img-6.8.0-100-generic
15 }
```

C.2 Konfiguration von ttyd des Hauptsystems

```
1 # /etc/default/ttyd
2
3 TTYD_OPTIONS="-i 0.0.0.0 -p 7681 -O -W login"
```

C.3 Cronjob des Root-Nutzers

```
1 [ ROOT CRONJOBS ]
2 * * * * * /usr/local/bin/nginx-healer.sh > /dev/null 2>&1
3 25 15 * * * /usr/sbin/shutdown -h now > /dev/null 2>&1
```

C.4 Auto-Healer-Script (nginx-autohealer.sh)

```
1 #!/bin/bash
2 # Nginx Watchdog & Auto-Healer V2 (Closed-Loop Edition)
3 # Prüft, ob Nginx läuft. Wenn nicht, wird die Konfiguration getestet.
4 # Neu: Testet isolierte (.broken) Dateien auf erfolgreiche manuelle Reparatur.
5 # Regeneriert komplette Verzeichnisstrukturen (Superset-Methode NodeJS/PHP) oder
6   isoliert fehlerhafte vHosts.
7 # Manueller Aufruf: ./nginx-healer.sh --force (oder -f)
8
9 # --- PARAMETER CHECK (MANUAL OVERRIDE) ---
10 FORCE_RUN=0
11 if [[ "$1" == "--force" ]] || [[ "$1" == "-f" ]]; then
12     FORCE_RUN=1
13     echo "Nginx Auto-Heal: Manueller Override aktiv."
14     logger -p syslog.info "Nginx Auto-Heal: Skript wurde manuell mit --force
15 gestartet."
16 fi
17
18 # --- STUFE 0: AUTO-RESTORE (Closed-Loop System) ---
19 # Prüft bei JEDEM Durchlauf, ob der User im CloudPanel seine Fehler behoben hat.
20 BROKEN_FILES=$(ls -l /etc/nginx/sites-enabled/*.broken 2>/dev/null)
21
22 if [ -n "$BROKEN_FILES" ]; then
23     if [ $FORCE_RUN -eq 1 ]; then echo "Prüfe isolierte (.broken) Dateien auf
24 Reparatur..."; fi
25
26     for b_file in $BROKEN_FILES; do
27         ORIG_FILE="${b_file%.broken}"
28
29         # Datei testweise wieder scharfschalten
30         mv "$b_file" "$ORIG_FILE"
31
32         # Trockenübung: Ist die Konfiguration jetzt valide?
33         if nginx -t >/dev/null 2>&1; then
34             logger -p syslog.info "Nginx Auto-Heal: vHost erfolgreich repariert und
35 reintegriert: $ORIG_FILE"
36             if [ $FORCE_RUN -eq 1 ]; then echo "-> ERFOLG: $ORIG_FILE ist wieder
37 fehlerfrei und aktiv."; fi
38
39             # Nginx direkt anweisen, die neue, fehlerfreie Config in den RAM zu laden
40             systemctl reload nginx
41         else
42             # Reparatur war erfolglos (User hat den Fehler noch nicht behoben) ->
43 Wieder isolieren
44             mv "$ORIG_FILE" "$b_file"
45             if [ $FORCE_RUN -eq 1 ]; then echo "-> FEHLGESCHLAGEN: $b_file ist immer
46 noch defekt. Bleibt isoliert."; fi
47         fi
48     done
49 fi
50
51 # --- HAUPT-BEDINGUNG (Watchdog) ---
52 # Startet, wenn Nginx tot ist ODER wenn das Skript manuell erzwungen wird
53 if [ $FORCE_RUN -eq 1 ] || ! systemctl is-active --quiet nginx; then
54
55     # Konfiguration testen und Ausgabe abfangen
56     TEST_OUTPUT=$(nginx -t 2>&1)
57
58     if echo "$TEST_OUTPUT" | grep -q "failed"; then
59
60         if [ $FORCE_RUN -eq 1 ]; then echo "Fehlerhafte Konfiguration erkannt. Starte
61 Heilungsprozess..."; fi
62
63         # --- STUFE 1: PRE-EMPTIVE AUTO-KORREKTUR (Full Tree Regeneration - Superset)
64         ---
65     fi
66 fi
```

```

56 MISSING_PATH=$(echo "$TEST_OUTPUT" | grep -oP 'open\\(\\) "\\K[~"]+')
57
58 if [ -n "$MISSING_PATH" ] && echo "$TEST_OUTPUT" | grep -q "No such file or
59 directory"; then
60
61     MISSING_DIR=$(dirname "$MISSING_PATH")
62     MISSING_USER=$(echo "$MISSING_PATH" | awk -F/ '{print $3}') # Extrahiert
63     z.B. kito01
64
65     if [ -n "$MISSING_USER" ] && [[ "$MISSING_PATH" == /home/* ]]; then
66
67         # DAS FINALE SUPERSET-ARRAY: Kombination aus NodeJS und PHP/Varnish
68         declare -a REQUIRED_DIRS=(
69             "/home/$MISSING_USER/backups/databases"
70             "/home/$MISSING_USER/htdocs"
71             "/home/$MISSING_USER/logs/nginx"
72             "/home/$MISSING_USER/logs/php"
73             "/home/$MISSING_USER/logs/varnish-cache"
74             "/home/$MISSING_USER/tmp"
75             "/home/$MISSING_USER/.nvm"
76             "/home/$MISSING_USER/.ssh"
77             "/home/$MISSING_USER/.varnish-cache"
78         )
79
80         # 1. Spezifischen Ordner anlegen (Sicherheitsnetz)
81         mkdir -p "$MISSING_DIR"
82
83         # 2. Komplette Superset-Struktur wiederherstellen
84         for dir in "${REQUIRED_DIRS[@]"; do
85             if [ ! -d "$dir" ]; then
86                 mkdir -p "$dir"
87             fi
88         done
89
90         # 3. Berechtigungen (chown) rekursiv und für versteckte Ordner fixen
91         chown "$MISSING_USER":"$MISSING_USER" "/home/$MISSING_USER" 2>/dev/
92         null
93
94         # Standard- und Hidden-Ordner der Root-Ebene durchiterieren
95         for bdir in backups htdocs logs tmp .nvm .ssh .varnish-cache; do
96             if [ -d "/home/$MISSING_USER/$bdir" ]; then
97                 chown -R "$MISSING_USER":"$MISSING_USER" "/home/$MISSING_USER
98                 /$bdir" 2>/dev/null
99             fi
100         done
101
102         logger -p syslog.warn "Nginx Auto-Heal: Benutzer-Verzeichnisstruktur
103         (Superset) für '$MISSING_USER' regeneriert."
104         if [ $FORCE_RUN -eq 1 ]; then echo "-> Verzeichnisstruktur für '
105         $MISSING_USER' wiederhergestellt."; fi
106     else
107         mkdir -p "$MISSING_DIR"
108         logger -p syslog.warn "Nginx Auto-Heal: System-Verzeichnis '
109         $MISSING_DIR' neu erstellt."
110         if [ $FORCE_RUN -eq 1 ]; then echo "-> Verzeichnis '$MISSING_DIR'
111         wiederhergestellt."; fi
112     fi
113
114     # Nginx erneut testen, um zu sehen, ob das mkdir das Problem gelöst hat
115     TEST_OUTPUT=$(nginx -t 2>&1)
116 fi
117
118 # --- STUFE 2: ISOLATION (Wenn Auto-Korrektur nicht gereicht hat) ---
119 if echo "$TEST_OUTPUT" | grep -q "failed"; then

```



```

113 BROKEN_FILE=$(echo "$TEST_OUTPUT" | grep -oP 'in /etc/nginx/[~:]+ ' | head
114 -1 | sed 's/^in //' )
115
116 if [ -z "$BROKEN_FILE" ]; then
117     if [ -n "$MISSING_PATH" ]; then
118         BROKEN_FILE=$(grep -rl "$MISSING_PATH" /etc/nginx/sites-enabled/
119 2>/dev/null | head -1)
120     fi
121     if [ -z "$BROKEN_FILE" ] && [ -n "$MISSING_USER" ]; then
122         BROKEN_FILE=$(grep -rl "/home/$MISSING_USER/" /etc/nginx/
123 sites-enabled/ 2>/dev/null | head -1)
124     fi
125 fi
126
127 if [ -n "$BROKEN_FILE" ] && [ -f "$BROKEN_FILE" ]; then
128     mv "$BROKEN_FILE" "${BROKEN_FILE}.broken"
129     logger -p syslog.err "Nginx Auto-Heal: vHost isoliert. Datei '
130 $BROKEN_FILE' wurde deaktiviert."
131     if [ $FORCE_RUN -eq 1 ]; then echo "-> vHost isoliert: $BROKEN_FILE
132 wurde deaktiviert."; fi
133 else
134     logger -p syslog.err "Nginx Auto-Heal: Konfiguration defekt. Konnte
135 fehlerhafte Datei nicht isolieren."
136     if [ $FORCE_RUN -eq 1 ]; then echo "-> KRITISCH: Konnte fehlerhafte
137 Datei nicht isolieren!"; fi
138 fi
139
140 if [ -n "$BROKEN_FILE" ] && [ -f "$BROKEN_FILE" ]; then
141     mv "$BROKEN_FILE" "${BROKEN_FILE}.broken"
142     logger -p syslog.err "Nginx Auto-Heal: vHost isoliert. Datei '
143 $BROKEN_FILE' wurde deaktiviert."
144     if [ $FORCE_RUN -eq 1 ]; then echo "-> vHost isoliert: $BROKEN_FILE
145 wurde deaktiviert."; fi
146 else
147     logger -p syslog.err "Nginx Auto-Heal: Konfiguration defekt. Konnte
148 fehlerhafte Datei nicht isolieren."
149     if [ $FORCE_RUN -eq 1 ]; then echo "-> KRITISCH: Konnte fehlerhafte
150 Datei nicht isolieren!"; fi
151 fi
152
153 # --- FINALES RESTART/RELOAD PROTOKOLL ---
154 logger -p syslog.info "Nginx Auto-Heal: Wende Systemstatus an (Reload/Restart)...
155 "
156
157 # Nutzt reload-or-restart: Macht einen sanften Reload, wenn Nginx läuft (keine
158 Downtime!), oder einen Start, wenn er tot ist.
159 systemctl reload-or-restart nginx
160
161 if systemctl is-active --quiet nginx; then
162     logger -p syslog.info "Nginx Auto-Heal: Nginx läuft erfolgreich. System
163 online."
164     if [ $FORCE_RUN -eq 1 ]; then echo "DONE: Nginx läuft erfolgreich."; fi
165 else
166     logger -p syslog.err "Nginx Auto-Heal: Start/Reload fehlgeschlagen. Manueller
167 Eingriff erforderlich."
168     if [ $FORCE_RUN -eq 1 ]; then echo "FEHLER: Nginx konnte nicht gestartet/
169 geladen werden."; fi
170 fi
171
172 else
173     # Auskommentieren, wenn die Testphase vorbei ist! (Vermeidet Log-Spam im Syslog)
174     # logger -p syslog.debug "Nginx Auto-Heal: Nginx läuft fehlerfrei. Nichts zu tun
175 ."
176     if [ $FORCE_RUN -eq 1 ]; then echo "Nginx Auto-Heal: Nginx läuft fehlerfrei. Ich
177 habe nichts zu tun."; fi
178 fi

```

C.5 Service-Script für den Factory-Reset (sda2-factory-reset.service)

```
1 [Unit]
2 Description=Factory Reset Visual Interface
3 After=local-fs.target
4 DefaultDependencies=no
5
6 [Service]
7 Type=oneshot
8 ExecStart=/usr/local/bin/factory-restore.sh
9 StandardOutput=tty
10 StandardError=tty
11 TTYPath=/dev/tty1
12 TTYReset=yes
13 TTYVHangup=yes
14
15 [Install]
16 WantedBy=basic.target
```

C.6 Shell-Script für den Factory-Reset

```
1#!/bin/bash
2
3# 1. Trigger-Check
4if ! grep -q "factory-reset" /proc/cmdline; then
5    exit 0
6fi
7
8# 2. UI-Vorbereitung (Ausgabe auf Konsole)
9exec > /dev/tty1 2>&1
10clear
11echo "===== "
12echo "          FACTORY RESET AKTIVIERT          "
13echo "===== "
14echo ""
15
16# 3. Variablen
17IMAGE="/root/clean-state.img.gz"
18TARGET="/dev/sda3"
19
20# 4. Validierung
21if [ ! -f "$IMAGE" ]; then
22    echo "[FEHLER] Quelldatei $IMAGE nicht gefunden!"
23    sleep 10
24    reboot
25fi
26
27echo "[1/2] Bereite Zielpartition $TARGET vor..."
28umount $TARGET 2>/dev/null
29
30echo "[2/2] Schreibe System-Image zurück..."
31echo "      Bitte warten, das System startet danach neu."
32echo "-----"
33
34# 5. Der Kern-Befehl
35# -W (Wide mode) sorgt bei partclone oft für eine bessere Anzeige in der Konsole.
36zcat "$IMAGE" | pv -pbt | partclone.ext4 -r -o "$TARGET"
37echo "-----"
38echo "SYNC: Daten werden auf Festplatte geschrieben..."
39sync
40echo ""
41echo "FERTIG! Das System ist wieder im Werkszustand."
42echo "Neustart in 5 Sekunden..."
43sleep 5
44reboot
```


C.7 Standard-Konfiguration der LXC-Container (default-profil.yaml)

```
1 name: default
2 description: Default LXD profile
3 devices:
4   eth0:
5     network: macvlan0
6     type: nic
7   eth1:
8     network: lxdbr0
9     type: nic
10  gpu-1:
11    gputype: physical
12    pci: '0000:00:02.0'
13    type: gpu
14 config:
15   limits.cpu: '4'
16   limits.memory: 6GiB
17   security.nesting: 'true'
18   boot.autostart: 'true'
19   boot.autostart.delay: '10'
20   boot.host_shutdown_timeout: '10'
21 access_entitlements:
22   - can_delete
23   - can_edit
24 project: default
```

C.8 Konfigurationsdatei der Netzwerkbrücke (networks-lxdbr0.yaml)

```
1 access_entitlements:
2   - can_edit
3   - can_delete
4 project: default
5 name: lxdbr0
6 description: ''
7 type: bridge
8 config:
9   ipv4.address: 192.168.212.1/24
10  ipv4.nat: 'false'
11  ipv6.address: none
```

C.9 Konfigurationsdatei des Docker-Containers (docker-sandbox.yaml)

```
1 access_entitlements:
2   - can_access_console
3   - can_delete
4   - can_edit
5   - can_exec
6   - can_manage_backups
7   - can_manage_snapshots
8   - can_update_state
9 name: docker-sandbox
10 description: ''
11 status: Running
12 status_code: 103
13 created_at: '2026-03-03T12:24:54.755796335Z'
14 last_used_at: '2026-03-04T08:41:54.290170021Z'
15 location: none
16 type: container
17 project: default
18 architecture: x86_64
19 ephemeral: false
20 stateful: false
21 profiles:
```

```

22 - default
23 config:
24   cloud-init.network-config: |+
25     version: 2
26     ethernet:
27       eth0:
28         dhcp4: true
29       eth1:
30         dhcp4: false
31         addresses:
32           - 192.168.212.2/24
33
34   image.architecture: amd64
35   image.description: ubuntu 24.04 LTS amd64 (release) (20260225)
36   image.label: release
37   image.os: ubuntu
38   image.release: noble
39   image.serial: '20260225'
40   image.type: squashfs
41   image.version: '24.04'
42   volatile.base_image: 48
43     da73f5dbaa33d365862df5c0798626cea4653a3a3cf41f557e32a8d878ef5e
44   volatile.cloud-init.instance-id: 8d6781ac-e88f-41af-923c-f248aa1c75c7
45   volatile.eth0.host_name: mace3a5c590
46   volatile.eth0.hwaddr: 00:16:3e:1b:80:fd
47   volatile.eth0.last_state.created: 'false'
48   volatile.eth0.name: eth0
49   volatile.eth1.host_name: vethc2d0f9ea
50   volatile.eth1.hwaddr: 00:16:3e:be:42:30
51   volatile.eth1.name: eth1
52   volatile.idmap.base: '0'
53   volatile.idmap.current: '[{"Isuid":true,"Isgid":false,"Hostid":1000000,"Nsid":0,"
54     Maprange":1000000000}, {"Isuid":false,"Isgid":true,"Hostid":1000000,"Nsid":0,"
55     Maprange":1000000000}]'
56   volatile.idmap.next: '[{"Isuid":true,"Isgid":false,"Hostid":1000000,"Nsid":0,"
57     Maprange":1000000000}, {"Isuid":false,"Isgid":true,"Hostid":1000000,"Nsid":0,"
58     Maprange":1000000000}]'
59   volatile.last_state.idmap: '[]'
60   volatile.last_state.power: RUNNING
61   volatile.uuid: cd421383-0d9d-4b44-8109-02bef7421d62
62   volatile.uuid.generation: da136ab8-9917-4462-95af-4b9cd36705f9
63 devices:
64   root:
65     path: /
66     pool: root
67     type: disk

```

C.10 Konfiguration von ttyd des LXC-Containers

```

1 # /etc/default/ttyd
2
3 TTYD_OPTIONS="-i 0.0.0.0 -p 7681 -O -W login"

```

C.11 Ausschnitt der PHP-Datei des Dashboards (index.php)

```
1 <?php
2 // --- SICHERHEITS-KONFIGURATION (SUDOERS) -----
3 /*
4 DER FINALE "INGO-FIX" FÜR UBUNTU 24.04 & CLOUDPANEL:
5
6 1. Klicke im Dashboard auf die 'Diagnose' Kachel (Stethoskop-Icon).
7 2. Notiere den Namen hinter "PHP USER IST".
8 3. Editiere die sudoers Datei auf dem Host: sudo visudo
9 4. Füge ganz unten folgende Zeile ein (ersetze [USER] durch den Namen aus der
   Diagnose):
10
11 [USER] ALL=(ALL) NOPASSWD: /usr/sbin/grub-reboot, /usr/bin/systemctl reboot, /usr/
   sbin/lxc, /usr/bin/systemctl, /usr/bin/docker, /usr/sbin/reboot, /usr/bin/
   journalctl, /usr/bin/lspci, /usr/bin/ls, /usr/sbin/nginx, /usr/bin/crontab
12
13 5. Speichern (Strg+O, Enter, Strg+X). Danach fließen auch die Recovery-Logs live!
14 */
15
16 // --- ZENTRALE KONFIGURATION -----
17 $config = [
18     'cloudpanel' => [
19         'name' => 'CloudPanel',
20         'url' => 'https://sandbox.ma.ki',
21         'port' => 8443,
22         'user' => 'admin',
23         'pass' => 'IT123_maki'
24     ],
25     'lxd_web' => [
26         'name' => 'LXD Web UI',
27         'url' => 'https://sandbox.ma.ki',
28         'port' => 9443,
29         'info' => 'Anmeldung über Browser-Zertifikat'
30     ],
31     'portainer' => [
32         'name' => 'Portainer',
33         'url' => 'https://docker-sandbox.ma.ki',
34         'port' => 9443,
35         // BYPASS MACVLAN: Pingt für den Status direkt die feste Bridge-IP an!
36         'check_host' => '192.168.212.2',
37         'user' => 'admin',
38         'pass' => 'IT123456_maki'
39     ],
40     'host_ssh' => [
41         'name' => 'Host System (Bare Metal)',
42         'user' => 'web',
43         'pass' => 'IT123_maki',
44         'info' => 'Sudo Privilegien aktiv'
45     ],
46     'lxc_ssh' => [
47         'name' => 'LXC Container (Docker Engine)',
48         'user' => 'root',
49         'pass' => 'IT123_maki',
50         'info' => 'Direktzugriff als Root-User'
51     ],
52     'ttyd_host' => [
53         'name' => 'Host Terminal',
54         'url' => 'https://web-cli.sandbox.ma.ki',
55         'port' => 443
56     ],
57     'ttyd_docker' => [
58         'name' => 'Docker Terminal',
59         'url' => 'https://docker-cli.sandbox.ma.ki',
60         'port' => 443,
61         // BYPASS PROXY & MACVLAN: Pingt den direkten Daemon-Port über die Bridge
```

```

62         'check_host' => '192.168.212.2',
63         'check_port' => 7681
64     ],
65     'recovery_grub_id' => '4',
66     'lxc_docker_name'   => 'docker-sandbox',
67     'lxc_snapshot'     => 'clean-state',
68     'main_domain'      => 'sandbox.ma.ki'
69 ];
70
71 /**
72  * Hilfsfunktion zur Konstruktion der vollständigen URL mit Port
73  */
74 function buildFullUrl($item) {
75     if (!isset($item['url'])) return '';
76     $port = isset($item['port']) ? ':' . $item['port'] : '';
77     return $item['url'] . $port;
78 }
79
80 // --- LIVE STATS & SERVICE LOGIC -----
81 function checkServiceStatus($item) {
82     $host = isset($item['check_host']) ? $item['check_host'] : parse_url($item['url'], PHP_URL_HOST);
83     $port = isset($item['check_port']) ? $item['check_port'] : $item['port'];
84
85     if (!$host) return false;
86
87     // Timeout auf 0.8s gesetzt, falls die Bridge Latenz hat
88     $connection = @fsockopen($host, $port, $errno, $errstr, 0.8);
89     if (is_resource($connection)) {
90         fclose($connection);
91         return true;
92     }
93     return false;
94 }
95
96 function getLiveStats($config) {
97     function readCpuStats() {
98         $data = @file('/proc/stat');
99         $cores = [];
100         if($data) {

```

C.12 Ausschnitt der Design-Anpassungen (CSS)

```

1  /*! tailwindcss v4.2.1 | MIT License | https://tailwindcss.com */
2  @layer properties;
3  @layer theme, base, components, utilities;
4
5  @layer theme {
6
7      :root,
8      :host {
9          --font-sans: ui-sans-serif, system-ui, sans-serif, "Apple Color Emoji",
10             "Segoe UI Emoji", "Segoe UI Symbol", "Noto Color Emoji";
11          --font-mono: ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas, "Liberation
12             Mono",
13             "Courier New", monospace;
14          --color-red-500: oklch(63.7% 0.237 25.331);
15          --color-red-600: oklch(57.7% 0.245 27.325);
16          --color-red-900: oklch(39.6% 0.141 25.723);
17          --color-red-950: oklch(25.8% 0.092 26.042);
18          --color-orange-400: oklch(75% 0.183 55.934);
19          --color-orange-500: oklch(70.5% 0.213 47.604);
20          --color-orange-600: oklch(64.6% 0.222 41.116);
21          --color-orange-900: oklch(40.8% 0.123 38.172);
22          --color-amber-400: oklch(82.8% 0.189 84.429);

```

```

22  --color-amber-500: oklch(76.9% 0.188 70.08);
23  --color-green-500: oklch(72.3% 0.219 149.579);
24  --color-emerald-500: oklch(69.6% 0.17 162.48);
25  --color-cyan-400: oklch(78.9% 0.154 211.53);
26  --color-cyan-500: oklch(71.5% 0.143 215.221);
27  --color-blue-400: oklch(70.7% 0.165 254.624);
28  --color-blue-500: oklch(62.3% 0.214 259.815);
29  --color-blue-600: oklch(54.6% 0.245 262.881);
30  --color-purple-400: oklch(71.4% 0.203 305.504);
31  --color-purple-500: oklch(62.7% 0.265 303.9);
32  --color-slate-200: oklch(92.9% 0.013 255.508);
33  --color-slate-300: oklch(86.9% 0.022 252.894);
34  --color-slate-400: oklch(70.4% 0.04 256.788);
35  --color-slate-500: oklch(55.4% 0.046 257.417);
36  --color-slate-600: oklch(44.6% 0.043 257.281);
37  --color-slate-700: oklch(37.2% 0.044 257.287);
38  --color-slate-800: oklch(27.9% 0.041 260.031);
39  --color-slate-900: oklch(20.8% 0.042 265.755);
40  --color-slate-950: oklch(12.9% 0.042 264.695);
41  --color-black: #000;
42  --color-white: #fff;
43  --spacing: 0.25rem;
44  --container-6xl: 72rem;
45  --container-7xl: 80rem;
46  --text-xs: 0.75rem;
47  --text-xs--line-height: calc(1 / 0.75);
48  --text-sm: 0.875rem;
49  --text-sm--line-height: calc(1.25 / 0.875);
50  --text-base: 1rem;
51  --text-base--line-height: calc(1.5 / 1);
52  --text-lg: 1.125rem;
53  --text-lg--line-height: calc(1.75 / 1.125);
54  --text-xl: 1.25rem;
55  --text-xl--line-height: calc(1.75 / 1.25);
56  --text-2xl: 1.5rem;
57  --text-2xl--line-height: calc(2 / 1.5);
58  --text-4xl: 2.25rem;
59  --text-4xl--line-height: calc(2.5 / 2.25);
60  --font-weight-bold: 700;
61  --font-weight-extrabold: 800;
62  --font-weight-black: 900;
63  --tracking-tighter: -0.05em;
64  --tracking-tight: -0.025em;
65  --tracking-normal: 0em;
66  --tracking-wider: 0.05em;
67  --tracking-widest: 0.1em;
68  --leading-relaxed: 1.625;
69  --radius-lg: 0.5rem;
70  --radius-xl: 0.75rem;
71  --radius-2xl: 1rem;
72  --radius-3xl: 1.5rem;
73  --animate-pulse: pulse 2s cubic-bezier(0.4, 0, 0.6, 1) infinite;
74  --default-transition-duration: 150ms;
75  --default-transition-timing-function: cubic-bezier(0.4, 0, 0.2, 1);
76  --default-font-family: var(--font-sans);
77  --default-mono-font-family: var(--font-mono);
78  }
79  }
80
81  @layer base {
82
83      *,
84      ::after,
85      ::before,
86      ::backdrop,
87      ::file-selector-button {

```

```

88     box-sizing: border-box;
89     margin: 0;
90     padding: 0;
91     border: 0 solid;
92 }
93
94 html,
95 :host {
96     line-height: 1.5;
97     -webkit-text-size-adjust: 100%;
98     tab-size: 4;
99     font-family: var(--default-font-family, ui-sans-serif, system-ui, sans-serif, "
Apple Color Emoji", "Segoe UI Emoji", "Segoe UI Symbol", "Noto Color Emoji");
100     font-feature-settings: var(--default-font-feature-settings, normal);
101     font-variation-settings: var(--default-font-variation-settings, normal);
102     -webkit-tap-highlight-color: transparent;
103 }
104
105 hr {
106     height: 0;
107     color: inherit;
108     border-top-width: 1px;
109 }
110
111 abbr:where([title]) {
112     -webkit-text-decoration: underline dotted;
113     text-decoration: underline dotted;

```

C.13 vHost des Reverse-Proxies Hauptsystem-TTYd

```

1 server {
2     listen 80;
3     listen [::]:80;
4     listen 443 quic;
5     listen 443 ssl;
6     listen [::]:443 quic;
7     listen [::]:443 ssl;
8     http2 on;
9     http3 off;
10    {{ssl_certificate_key}}
11    {{ssl_certificate}}
12    server_name web-cli.sandbox.ma.ki;
13    {{root}}
14
15    {{nginx_access_log}}
16    {{nginx_error_log}}
17
18    if ($scheme != "https") {
19        rewrite ^ https://$host$request_uri permanent;
20    }
21
22    location / {
23        proxy_pass http://sandbox.ma.ki:7681;
24        proxy_http_version 1.1;
25        proxy_set_header Host $host;
26
27        # WebSocket Support zwingend für ttyd
28        proxy_set_header Upgrade $http_upgrade;
29        proxy_set_header Connection "upgrade";
30        proxy_set_header X-Real-IP $remote_addr;
31        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
32        proxy_set_header X-Forwarded-Proto $scheme;
33
34        # CloudPanel's globale X-Frame-Options entfernen
35        proxy_hide_header X-Frame-Options;

```

```

36     fastcgi_hide_header X-Frame-Options;
37
38     # CSP setzen (Exakte URL des Dashboards!)
39     add_header Content-Security-Policy "frame-ancestors 'self' https://sandbox.ma.ki;
    " always;
40 }
41 }

```

C.14 vHost des Reverse-Proxies Container-TTYd

```

1 server {
2     listen 80;
3     listen [::]:80;
4     listen 443 quic;
5     listen 443 ssl;
6     listen [::]:443 quic;
7     listen [::]:443 ssl;
8     http2 on;
9     http3 off;
10    {{ssl_certificate_key}}
11    {{ssl_certificate}}
12    server_name docker-cli.sandbox.ma.ki;
13    {{root}}
14
15    {{nginx_access_log}}
16    {{nginx_error_log}}
17
18    if ($scheme != "https") {
19        rewrite ^ https://$host$request_uri permanent;
20    }
21
22    location / {
23        proxy_pass http://192.168.212.2:7681;
24        proxy_http_version 1.1;
25        proxy_set_header Host $host;
26
27        # WebSocket Support zwingend für ttyd
28        proxy_set_header Upgrade $http_upgrade;
29        proxy_set_header Connection "upgrade";
30        proxy_set_header X-Real-IP $remote_addr;
31        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
32        proxy_set_header X-Forwarded-Proto $scheme;
33
34        # CloudPanel's globale X-Frame-Options entfernen
35        proxy_hide_header X-Frame-Options;
36        fastcgi_hide_header X-Frame-Options;
37
38        # CSP setzen (Exakte URL des Dashboards!)
39        add_header Content-Security-Policy "frame-ancestors 'self' https://sandbox.ma.ki;
    " always;
40    }
41 }

```

D Anlagen - Hostnamen, IP-Adressen, Initial-Passwörter

Tabelle 2: Detaillierte Dienst- und Portmatrix der Sandbox

Dienst	IP-Adresse	Port	URL
Web-Server (Hub)	192.168.100.21	443	https://sandbox.ma.ki/
CloudPanel UI	192.168.100.21	8443	https://sandbox.ma.ki:8443/
LXD Dashboard	192.168.100.21	9443	https://sandbox.ma.ki:9443/
SSH-Host (Hauptsystem)	192.168.100.21	22	ssh://web@sandbox.ma.ki

Tabelle 3: Initial-Passwörter

Dienst	Username	Passwort
Recovery Shell/SSH	agent	IT123_maki
Hauptsystem Shell/SSH	web	IT123_maki
Cloudpanel UI	admin	IT123_maki
Docker LXC Shell/SSH	root	IT123_maki
Portainer UI	admin	IT123456_maki

Quellenverzeichnis

- [1] Canonical Ltd. *Ubuntu Server Download*. 2026.
<https://ubuntu.com/download/server>
(abgerufen am 14.04.2026).
- [2] Canonical Ltd. *LXD - How to initialize*. 2026.
<https://documentation.ubuntu.com/lxd/latest/howto/initialize/>
(abgerufen am 14.04.2026).
- [3] Docker Inc. *Install Docker Engine on Ubuntu*. 2026.
<https://docs.docker.com/engine/install/ubuntu/>
(abgerufen am 14.04.2026).
- [4] CloudPanel. *CloudPanel - High-Performance PHP Control Panel*. 2026.
<https://cloudpanel.io>
(abgerufen am 14.04.2026).
- [5] OpenJS Foundation. *Node.js - Download and install*. 2026.
<https://nodejs.org/en/download>
(abgerufen am 14.04.2026).